T.C. ALTINBAŞ UNIVERSITY
Software Engineering Department

SWE381 EMBEDDED SYSTEM DESIGN PROJECT

Duck Hunt Game with Arduino UNO and Unity

GitHub Link: https://github.com/KaganxSahin/KagansDuckHuntGame

Wokwi Link: https://wokwi.com/projects/460911006135012353

Student Name: Kağan Şahin
Student Number: 220513375

I declare that this report and project are my own individual work. I understand that if I submit the same project as another student, both of us will receive zero.

Student Signature: _____ **Date:** _____

Project Advisor: Asst. Prof. Dr. Fatma Kuzey Edeş Huyal

Istanbul-2026

Table of Contents

1. Introduction **3**
2. Use Case and Motivation **4**
3. System Design and Components **5**
3.1 General Architecture 5
3.2 Hardware Subsystem 5
3.3 Software Subsystem (Unity) 6
4. Circuit and Hardware Structure **6**
5. Working Principle **7**
6. Arduino Code and Detailed Discussion **8**
6.1 Global Declarations and Pin Assignment 8
6.2 The setup() Function 9
6.3 The loop() Function 10
6.4 The runMotor() Function — Motor Timing State Machine 12
7. Unity Code / Software Explanation **14**
8. Concepts Used in the Project **14**
8.1 delay() 14
8.2 Serial Communication 15
8.3 State-Based Logic 15
9. Delay vs millis() vs Finite State Machines **16**
9.1 Definitions 16
9.2 Comparison 16
9.3 Approach Used in This Project 17
10. Real Experiment and Wokwi Simulation **18**
10.1 Physical Hardware Implementation 18
10.2 Wokwi Simulation 19
11. Challenges and Hurdles **20**
11.1 Ambient Light Interference 20
11.2 Serial Communication Synchronisation 21
11.3 Motor Timing Without Real-Time Clock 21
11.4 Buffer Size and Data Formatting 21
12. Conclusion **22**
13. References **24**

1. Introduction

Duck Hunt Game with Arduino UNO and Unity project is a hardware-in-the-loop interactive game system that faithfully reimagines the classic Nintendo Duck Hunt experience of 1984 using modern, accessible embedded electronics and game development tools. At its core, the system comprises a physical light gun peripheral — constructed with an Arduino Uno microcontroller, a Light Dependent Resistor (LDR) sensor, a mechanical trigger switch, and a haptic vibration motor — that communicates in real time with a Unity 2D game running on a personal computer. The combination of these two domains produces an experience that is simultaneously nostalgic in its play style and contemporary in its technical implementation.

The system operates on the principle of sequential light detection, an optical technique first used in 1984. When the player pulls the physical trigger, the Arduino detects the event and transmits a signal to Unity via a serial communication link. Unity temporarily blacks out the game screen and renders a white flash exclusively at the target's location. If the LDR, held pointed at the screen, detects that white flash, a hit is registered; otherwise, a miss is recorded. The Arduino simultaneously drives a vibration motor through an L9110S dual H-bridge driver to provide tactile haptic recoil feedback at the moment of firing.

This project was selected because it elegantly unifies the foundational competencies of an embedded systems course: digital input reading, output actuation, serial communication, interrupt-free timing design, and hardware-software co-design. Rather than confining embedded logic to a self-contained device, the project demands that the microcontroller serve as a real-time peripheral to a complex software environment, creating a rich engineering challenge that extends well beyond simple sensor-actuator tasks.

2. Use Case and Motivation

The primary use case of this system is interactive gaming using a physical input peripheral that replaces conventional input devices such as a mouse or keyboard. In the context of the project, the "use case" is a single-player duck hunting game in which the user aims a physical gun-shaped device at a monitor and fires at animated duck targets. The system is applicable in scenarios such as arcade-style gaming installations, interactive museum exhibits, and educational demonstrations of sensor integration and game engine communication.

From a broader engineering perspective, the concepts demonstrated by this project have real-world parallels in industrial and consumer product design. The principle of using an optical sensor to detect a screen-rendered stimulus is directly analogous to barcode scanners and optical positioning systems. The haptic motor feedback mechanism is equivalent in principle to the rumble actuators in modern game controllers. The serial communication protocol between a microcontroller and a host application mirrors the design of many embedded peripheral devices, from medical instrumentation interfaces to industrial human-machine interfaces.

The motivation for combining Arduino and Unity specifically stems from the complementary strengths of each platform. Arduino provides low-level, deterministic hardware control with precise digital input reading and direct output actuation, tasks for which a general-purpose operating system would introduce unacceptable latency or non-determinism. Unity, conversely, provides a high-level, feature-rich environment for rendering animated graphics, managing game state, and handling display logic, none of which are practical on a microcontroller. The serial interface between them acts as a well-defined contract: the Arduino reports hardware states, and Unity acts upon them. This separation of concerns mirrors the layered architecture found in professional embedded system designs.

3. System Design and Components

3.1 General Architecture

The system architecture follows a client-server model in which the Arduino functions as the hardware client — continuously sampling sensors and driving actuators — while Unity serves as the software server — managing game logic, display, and state. The two subsystems exchange data through a bidirectional serial link over a USB cable at a baud rate of 9600 bps. The Arduino transmits a comma-delimited string containing the trigger state and sensor state on every loop iteration, while Unity monitors the incoming serial stream and interprets these values to update game state accordingly.

The flow of information in the system proceeds as follows. The player physically pulls the trigger on the gun peripheral. The Arduino reads the resulting LOW signal on its digital trigger pin (inverted via pull-up resistor logic), formats it alongside the current LDR sensor reading, and transmits the pair to Unity. Unity receives this message, initiates the screen flash sequence, and reads the subsequent sensor report from the Arduino to determine whether a hit or miss occurred. The result updates the game score, duck animations, and other UI elements. Simultaneously, the Arduino's internal motor control routine activates the vibration motor for a bounded duration to simulate recoil.

3.2 Hardware Subsystem

The hardware subsystem is centered on an Arduino Uno microcontroller, which serves as the primary processing unit for all real-time input reading and output control. All sensor and actuator interfaces are connected to the digital pins of the Arduino to ensure reliable binary state detection. The LDR module is connected to Digital Pin 2 and outputs a HIGH signal when the ambient light exceeds its calibrated threshold, which corresponds to the white flash rendered by Unity on the monitor. The mechanical trigger switch is connected to Digital Pin 3 and employs pull-up resistor logic, meaning the pin reads HIGH at rest and LOW when the trigger is depressed; the firmware inverts this reading to produce an intuitive active-HIGH trigger state. The L9110S H-bridge motor driver is connected to Digital Pin 4 on its A-1A control input, with the A-1B input tied to ground, allowing unidirectional motor control via a single digital write. A micro vibration motor is connected directly to the Motor A output terminals of the L9110S driver.

3.3 Software Subsystem (Unity)

On the software side, the Unity project contains four primary C# scripts. The ComController class manages the serial port connection, initializing the appropriate COM port and baud rate, and continuously reading incoming serial data from the Arduino. The GameController class implements the core game loop, including the black frame masking and white circle rendering that form the detection flash sequence. The Duck class encapsulates target behaviour, including movement patterns and animation state transitions between flying, hit, and falling states. Finally, the ScoreObject class manages on-screen UI elements that provide visual feedback to the player. These scripts work together under Unity's component-based architecture, with the ComController feeding parsed sensor data into the GameController, which drives all other game systems.

4. Circuit and Hardware Structure

The physical circuit is constructed around the Arduino Uno and interfaces three external modules: an LDR digital sensor module, a push-button trigger switch module, and an L9110S dual H-bridge motor driver with an attached micro vibration motor. All modules share a common 5 V power rail sourced from the Arduino's regulated 5 V output pin and a common ground plane. This common-ground configuration is essential for ensuring signal integrity across all digital interfaces. The table below summarises the hardware wiring:

Component	Pin	Pin Type	Role in the System
LDR Sensor (DO)	D2	Digital Input	Detects white flash on screen to confirm hit
Switch / Trigger (S)	D3	Digital Input	Physical trigger — pull-up inverted logic
L9110S Driver (A-1A)	D4	Digital Output	Drives vibration motor for haptic recoil
L9110S Driver (A-1B)	GND	Ground	Reference ground for the motor driver
All VCC / V+ Pins	5V	Power	Shared 5 V supply for all modules
All GND / G Pins	GND	Ground	Shared common ground for the entire circuit

The LDR module used in this project exposes a digital output (DO) pin rather than an analogue output. Internally, the module contains a comparator circuit that compares the analogue voltage from the LDR with a reference voltage set by an onboard potentiometer. When the incident light level exceeds the threshold, the comparator output transitions to HIGH. This design was deliberately chosen over an analogue reading approach because it eliminates the need for analogRead() calls and ADC conversion

time, resulting in faster, more deterministic detection that is critical for the split-second timing of the light-gun mechanism.

The L9110S dual H-bridge driver was selected for motor control because it can drive motors at voltages up to 12 V while being directly controllable by the Arduino's 5 V digital output pins. By tying the A-1B input permanently to ground and controlling only the A-1A input on Digital Pin 4, a simplified unidirectional drive circuit is achieved. When Pin 4 is set HIGH, the motor spins, producing vibration; when Pin 4 is set LOW, the motor stops. This design avoids the need for separate logic-level shifters or transistor switching circuits, reducing component count and potential points of failure.

5. Working Principle

The system's operation can be decomposed into a five-step detection cycle that repeats each time the player fires the gun. Understanding this cycle is essential to appreciating the elegance of the optical detection method employed.

Step 1 — Trigger Input: The player depresses the physical trigger switch. The resulting electrical signal transitions the voltage on Digital Pin 3 from HIGH to LOW. The Arduino firmware applies logical inversion to produce an active-HIGH trigger state (triggerState = 1) and immediately formats a serial message for transmission to Unity.

Step 2 — Black Frame Masking: Upon receiving the trigger signal, Unity renders the entire game screen solid black for approximately one display frame (approximately 16 milliseconds at 60 Hz). This masking step is imperceptible to the human eye due to its brevity but ensures that the LDR reads a baseline darkness level before the target flash is rendered.

Step 3 — Target Highlighting: Within the same masked frame, Unity renders a solid white circle at the exact screen coordinates occupied by the duck target at that instant. No other element is drawn. If the gun barrel is aimed at the target location, the LDR will be exposed to this white flash.

Step 4 — LDR Verification: The Arduino reads the state of Digital Pin 2. If the gun is pointed at the target, the LDR detects the white flash and the pin reads HIGH (sensorState = 1). If the gun is pointed

away, the pin remains LOW (sensorState = 0). The Arduino transmits this sensor state to Unity in the next serial message.

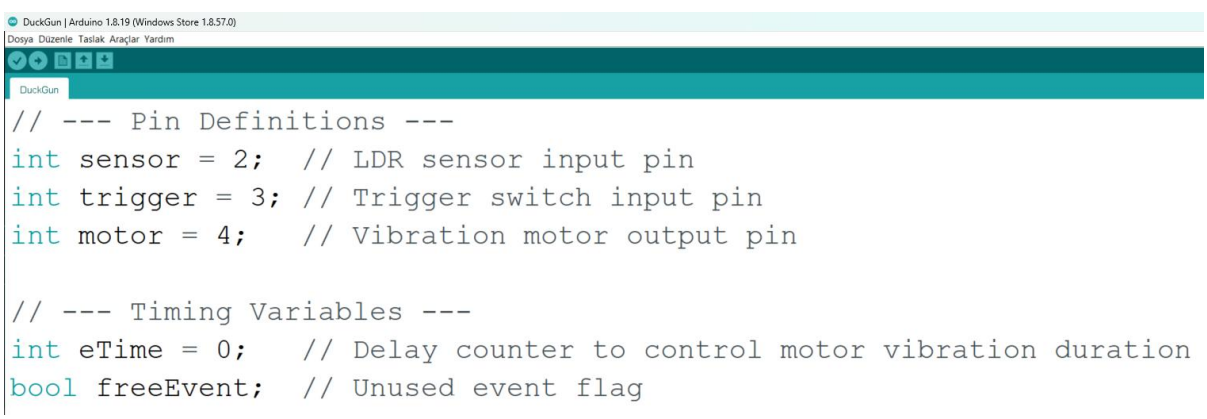
Step 5 — Validation and Feedback: Unity evaluates the combination of trigger and sensor states received within the same detection sequence. If both are HIGH, a hit is registered: the duck's animation transitions to a falling state, and the score is incremented. Simultaneously, the Arduino's runMotor() function activates the vibration motor via Pin 4 to provide haptic recoil. If the sensor state is LOW, a miss is recorded and the duck continues its flight path.

This entire cycle completes within a time frame of under 100 milliseconds, making the detection virtually instantaneous from the player's perspective. The serial communication link operates at 9600 bps and transmits short strings of the form "1,1" or "0,0", which are parsed by Unity's ComController with negligible processing overhead.

6. Arduino Code and Detailed Discussion

6.1 Global Declarations and Pin Assignment

The Arduino sketch begins with the declaration of three integer pin constants and two control variables. The pin assignments are as follows:




```
// --- Pin Definitions ---
int sensor = 2;  // LDR sensor input pin
int trigger = 3; // Trigger switch input pin
int motor = 4;   // Vibration motor output pin

// --- Timing Variables ---
int eTime = 0;   // Delay counter to control motor vibration duration
bool freeEvent;  // Unused event flag
```

The choice to use plain integers rather than const int declarations is a minor stylistic oversight that does not affect function; however, marking pin constants as const would prevent accidental reassignment. The variable eTime serves as a loop-iteration counter for the motor timing logic, not a real-time millisecond counter. This distinction is significant and is discussed in depth in Section 9. The bool

variable freeEvent is declared but never assigned or read in the provided code; it appears to be a placeholder for a future event-driven feature.

6.2 The setup() Function

The setup() function runs once at power-on or after a reset and is responsible for configuring the hardware peripherals:

```
void setup() {
  // Initialize pins
  pinMode(sensor, INPUT);
  pinMode(trigger, INPUT);
  pinMode(motor, OUTPUT);

  // Initialize serial communication
  Serial.begin(9600);
  while(!Serial); // Wait for serial connection to establish
}
```

The pinMode() calls configure the direction registers of the ATmega328P's I/O ports. Pins 2 and 3 are configured as inputs for the LDR and trigger respectively, while Pin 4 is configured as an output for the motor driver. Serial.begin(9600) initialises the UART peripheral with a baud rate of 9600 bits per second, consistent with the rate expected by Unity's ComController. The blocking call while(!Serial) waits until the serial port is ready, which is particularly important when running on boards that enumerate as virtual COM ports over USB, ensuring that no serial data is transmitted before the host application has established the connection. This is a best practice in Arduino-to-PC serial communication that prevents data loss during the initialisation window.

6.3 The loop() Function

The loop() function constitutes the main program cycle and executes repeatedly for the lifetime of the device:

```
void loop() {
  // Read inputs. Trigger is inverted (!) due to pull-up resistor logic.
  // Returns 1 when pressed, 0 when released.
  int triggerState = !digitalRead(trigger);

  // Read sensor state (1 or 0)
  int sensorState = digitalRead(sensor);

  // Format data as "trigger,sensor"
  char str[3];
  sprintf(str, "%d,%d", triggerState, sensorState);

  // Send data to Unity
  Serial.println(str);

  // Handle motor vibration logic
  runMotor();

  // Short delay for stability
  delay(1);
}
```

The first two lines read the current digital states of the trigger and sensor pins. The trigger reading is bitwise-NOT inverted using the logical NOT operator (!) because the switch module employs pull-up resistor logic: the pin is normally pulled HIGH and drops to LOW when the trigger is pressed. Inverting this produces the more intuitive convention where triggerState equals 1 when the trigger is pressed and 0 when at rest. The sensor reading requires no inversion because the LDR module's comparator output is already active-HIGH.

The sprintf() function formats these two integer values into a compact comma-delimited string stored in a three-character buffer. The format "%d,%d" produces strings such as "0,0", "1,0", or "1,1", which are all two or three characters in length. Serial.println() then transmits this string followed by a carriage return and newline ("\r\n"), which Unity's serial reader uses as a message delimiter. This framing approach is simple and reliable for short fixed-format messages.

It is worth noting that the char buffer str[3] is sized for the minimum required storage. The longest possible output of sprintf() with this format, "1,1", is three characters plus the null terminator, requiring at least four bytes. This is a potential buffer overflow if the format were ever extended; however, in the current design, with single-digit binary values, the buffer functions correctly.

The runMotor() function is called on every loop iteration to manage the motor state machine. The delay(1) at the end introduces a one-millisecond pause that serves two purposes: it stabilises the serial output rate to avoid flooding the host with data and it provides a consistent time base for the eTime counter used inside runMotor().

6.4 The runMotor() Function — Motor Timing State Machine

The runMotor() function implements the haptic feedback control logic and represents the most technically sophisticated portion of the Arduino sketch:

DuckGun | Arduino 1.8.19 (Windows Store 1.8.57.0)

Dosya Düzenle Taslak Araçlar Yardım

DuckGun

```
// --- Haptic Feedback Function ---
// Limits the vibration duration of the motor
void runMotor() {
  // Read inverted trigger state for motor logic
  int triggerState = !digitalRead(trigger);

  // Increment timer if vibration has started
  if (eTime > 0) {
     eTime++;
  }

  // Start motor if trigger is pressed and timer is idle
  if ((triggerState == 1) && (eTime == 0)) {
    eTime = 1;
    digitalWrite(motor, 1); // Turn ON motor
  }

  // Stop motor after 50 cycles (vibration duration)
  if (eTime == 50) {
    digitalWrite(motor, 0); // Turn OFF motor
  }

  // Reset timer after 80 cycles (cooldown period)
  if (eTime > 80) {
     eTime = 0;
  }
}
```

Derleme tamamlandı.

This function implements a four-state implicit state machine using the eTime counter as a proxy for elapsed time. The four operational phases are as follows:

Phase 1 — Idle (eTime == 0): The system awaits a trigger press. The motor is off and eTime remains at zero. This is the default resting state.

Phase 2 — Motor Active (1 <= eTime < 50): Upon detecting a trigger press with eTime at zero, eTime is set to 1 and the motor output pin is driven HIGH. On each subsequent call, eTime is incremented. Since each loop iteration has a guaranteed minimum duration of approximately one millisecond (due to the delay(1) call in loop()), eTime approximates elapsed milliseconds. The motor vibrates for approximately 50 milliseconds, producing a brief, sharp haptic pulse that simulates the recoil of a fired weapon.

Phase 3 — Motor Off, Cooldown Active (50 <= eTime <= 80): When eTime reaches exactly 50, the motor output is driven LOW, stopping the motor. eTime continues incrementing through the cooldown period. Critically, the condition (triggerState == 1) && (eTime == 0) cannot be true during cooldown because eTime is nonzero, preventing the motor from retriggering while the trigger is held. This implements a mandatory 30-cycle (approximately 30-millisecond) cooldown between firings.

Phase 4 — Reset (eTime > 80): Once eTime exceeds 80, it is reset to zero, returning the system to the idle state. The next trigger press will restart the cycle. The total motor cycle duration (50 ms active + 30 ms cooldown = 80 ms) was chosen to balance perceptible feedback with a rapid reset that allows quick successive shots.

This design choice — using a loop-iteration counter rather than millis() — is deliberate in its simplicity but introduces a dependency on a consistent loop execution time. The delay(1) at the end of loop() acts as a floor that guarantees each iteration takes at least one millisecond. However, if the serial transmission time or other code in the loop were to increase significantly, the effective time base of eTime would drift. For the relatively simple operations performed in this sketch, this approach is adequate and avoids the complexity of interrupt-driven timing.

7. Unity Code / Software Explanation

The Unity project serves as the game engine environment and is structured around four primary C# scripts. The ComController class is responsible for managing the serial port connection between the PC and the Arduino. It initialises the port using the correct COM port identifier and a matching baud rate of 9600, then continuously reads incoming data strings in a background thread or update loop to avoid blocking the main game thread. Parsed trigger and sensor values are exposed as public properties or events for consumption by other scripts.

The GameController class implements the central game logic, including the critical black frame and white circle flash sequence. When it receives a trigger signal from ComController, it transitions the screen to a black render state for one frame and draws a white circle at the duck target's current transform position. On the following frame, it reads the sensor state from ComController: a HIGH reading confirms a hit, while a LOW reading confirms a miss. This script also manages the game timer, round transitions, and dog animation sequences for misses.

The Duck class manages the autonomous behaviour of each target, including its flight trajectory, speed, and animation controller state transitions between idle, flying, hit, and falling states. Upon receiving a hit event from GameController, the Duck class triggers the appropriate falling animation and schedules its own destruction or recycling after a set delay. The ScoreObject class handles visual score popups and HUD updates, displaying point values at the location of a hit duck and updating the persistent score counter displayed at the bottom of the screen.

8. Concepts Used in the Project

8.1 delay()

The delay() function in Arduino suspends program execution for a specified number of milliseconds by busy-waiting in a tight loop, halting all code execution during that period. In this project, delay(1) is called at the end of each main loop iteration. Its purpose is twofold: it introduces a predictable one-millisecond pause that serves as the time base for the eTime motor counter, and it limits the serial transmission rate, preventing the output buffer from being overwhelmed and ensuring Unity receives data at a manageable rate. While delay() is straightforward to use, its blocking nature means that the microcontroller cannot respond to other events during the delay period. In this application, where all

inputs must be sampled every loop iteration, a one-millisecond delay is sufficiently short that no perceptible latency is introduced.

8.2 Serial Communication

Serial communication is the backbone of the hardware-software interface in this project. The Arduino uses its hardware UART peripheral, exposed through the Arduino Serial library, to transmit ASCII-encoded data frames to the PC at 9600 bps over the USB serial interface. Each data frame is a comma-delimited pair of binary integers terminated with a newline character, representing the current trigger and sensor states. Unity's ComController opens the corresponding virtual COM port and reads these frames, parsing the comma-separated values to extract individual state variables. This approach to inter-system communication is lightweight, widely supported, and requires no additional hardware beyond the USB connection already present for programming the Arduino. The serial link introduces a predictable latency on the order of a few milliseconds, which is well within the acceptable range for an interactive gaming application.

8.3 State-Based Logic

The motor control routine in runMotor() is an example of implicit state-based logic. The variable eTime encodes the current operational state of the motor subsystem, with specific numerical ranges corresponding to the idle, active, cooling, and reset states. Transitions between states are triggered by conditions evaluated on each function call: a trigger press with eTime == 0 triggers the active state, eTime == 50 triggers the off state, and eTime > 80 triggers the reset. This pattern, while not implemented as a formal enumerated finite state machine, embodies the same conceptual design and demonstrates the value of state-based reasoning in embedded firmware. State machines of this type are preferable to sequential blocking code because they do not prevent the main loop from executing other tasks while a timed sequence is in progress.

9. Delay vs millis() vs Finite State Machines

9.1 Definitions

delay(): A blocking function that halts all code execution for the specified duration. The CPU busy-waits, incrementing an internal timer until the duration elapses. Simple to use but prevents concurrent task execution during the wait.

millis(): A non-blocking function that returns the number of milliseconds elapsed since the Arduino was last reset. Used in conjunction with a stored timestamp to implement timed events: by comparing the current millis() value with the stored start time, elapsed time can be determined without halting the processor. Enables concurrent task scheduling within a single loop iteration.

Finite State Machine (FSM): A formal design pattern in which a system can exist in one of a finite number of states. Transitions between states are triggered by defined events or conditions. FSMs can be implemented with or without millis() — the state machine defines the logical structure, while millis() provides non-blocking timing for state durations.

9.2 Comparison

The primary advantage of delay() is its simplicity: it requires no additional variables and is immediately intuitive. Its fundamental disadvantage is that it blocks all processing, making it unsuitable for systems that must respond to external events during a wait period. If, for example, a trigger press occurred during a delay() call, it would be missed entirely.

The millis() approach resolves this limitation by decoupling time tracking from execution flow. A task that requires a 50-millisecond action can store the start time, check elapsed time on each loop iteration, and continue processing other inputs without pausing. This is the standard approach for responsive embedded firmware. Its disadvantage is slightly higher code complexity, requiring additional variables and conditional logic.

A Finite State Machine combines millis()-based timing with a formal state enumeration, producing the most readable and maintainable firmware architecture for complex multi-event systems. It adds conceptual overhead but greatly simplifies debugging and extension.

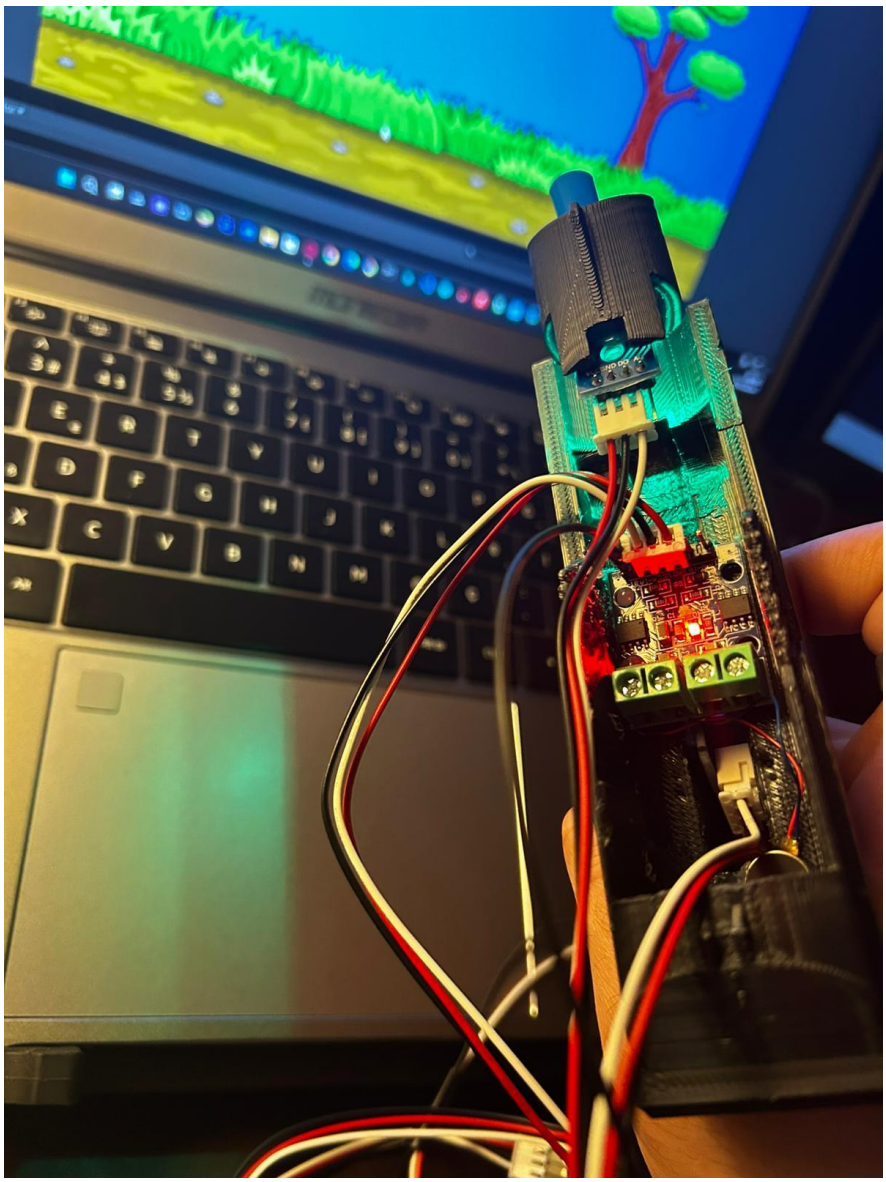
9.3 Approach Used in This Project

This project uses a hybrid approach. The main loop employs delay(1) for a simple, fixed-rate timing floor. The runMotor() function implements an implicit state machine using eTime as a loop-iteration counter, effectively approximating the behaviour of a millis()-based FSM without the overhead of multiple timestamp variables. This design is justified by the simplicity of the motor control task: only one timed sequence is required, and the loop execution time is sufficiently stable (guaranteed to be at least one millisecond by the delay) that eTime reliably approximates elapsed time. A full millis()-based FSM would be the recommended approach for a production system or one with more complex concurrent timing requirements.

10. Real Experiment and Wokwi Simulation

10.1 Physical Hardware Implementation

The physical build involves mounting the Arduino Uno, LDR module, trigger switch, and L9110S motor driver within a gun-shaped enclosure. The LDR is positioned at the tip of the barrel to maximise angular precision when aimed at a monitor. The trigger switch is mounted in the trigger guard position for ergonomic operation. The L9110S driver and micro vibration motor are housed in the grip of the enclosure. The Arduino Uno is connected to the PC via a standard USB-A to USB-B cable, which simultaneously powers the board and provides the virtual COM port for serial communication. The circuit is powered entirely from the Arduino's onboard 5 V regulator, which draws current from the USB host.



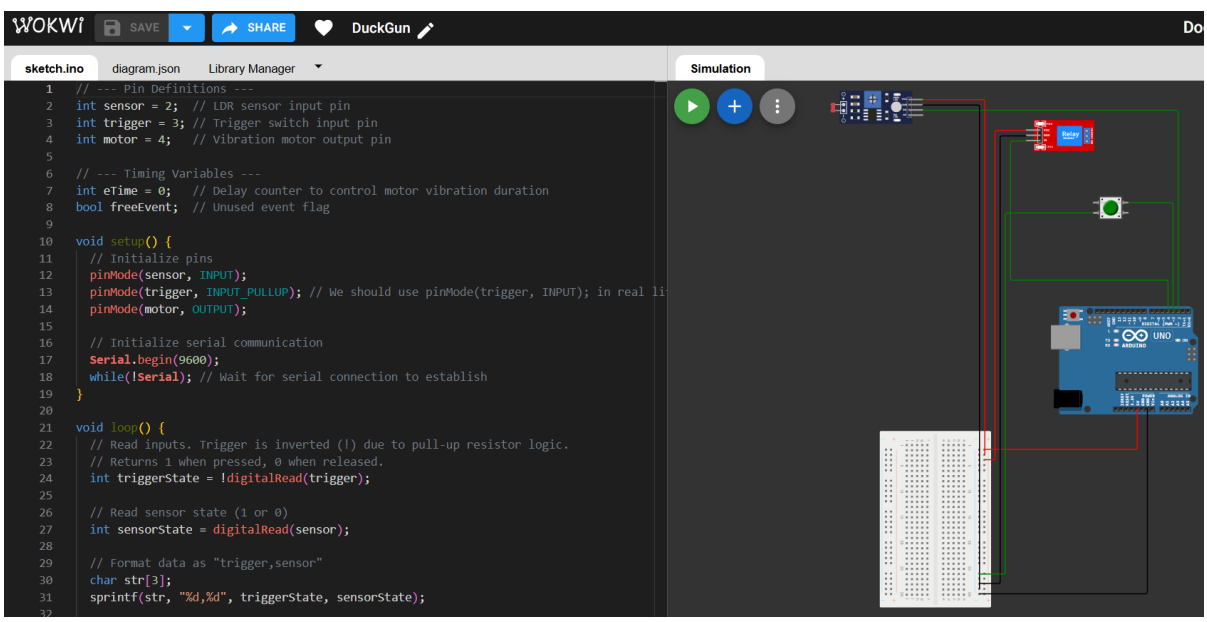
10.2 Wokwi Simulation

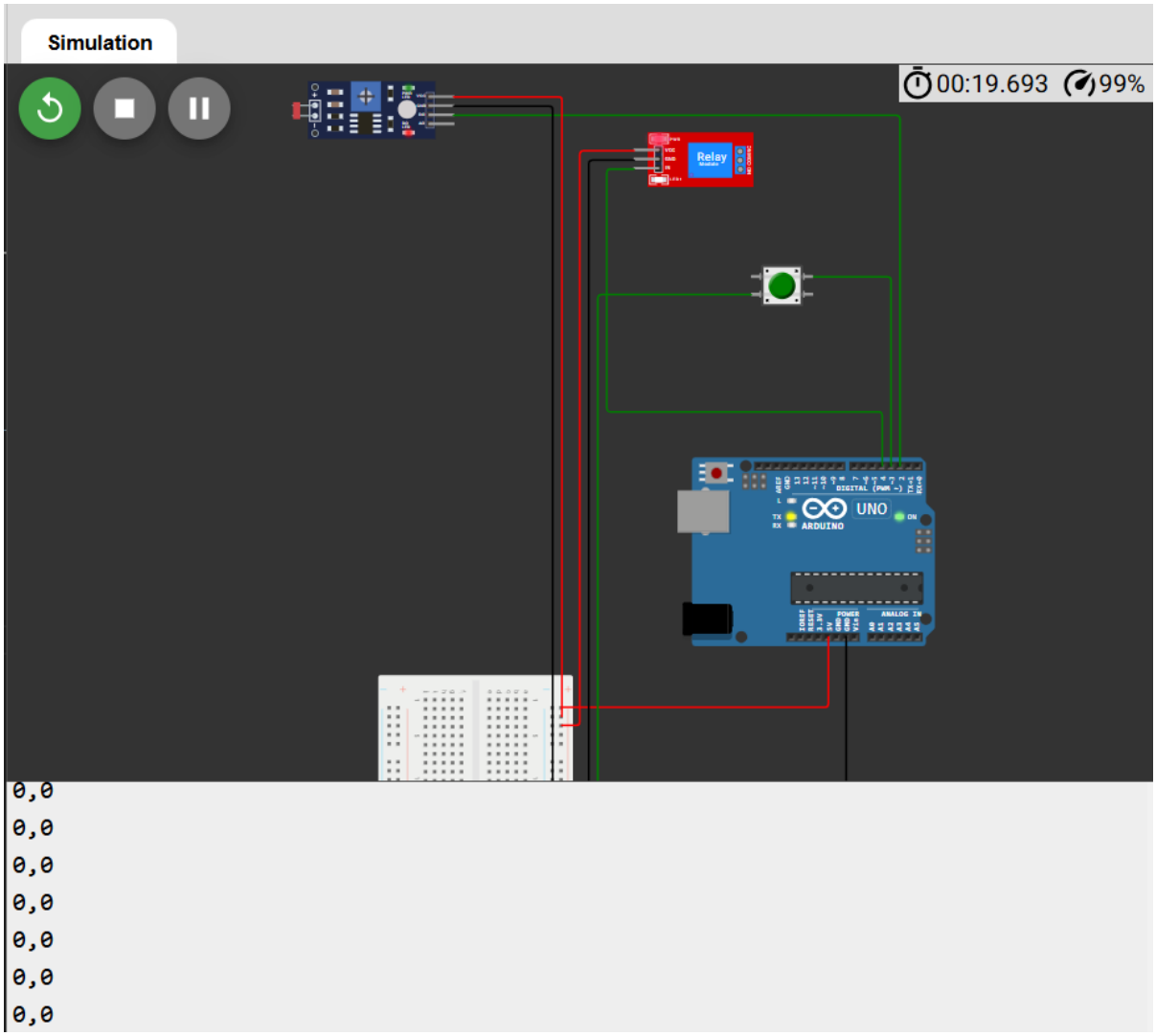
The Wokwi circuit simulation, accessible at the project repository, provides a virtual replication of the hardware circuit that can be tested without physical components. Because Wokwi does not include a native L9110S motor driver module or a haptic vibration motor, a relay module is used as a functional substitute. The relay is wired such that its coil is driven by the same digital output (Pin 4) that controls the motor driver in the physical build. When a successful hit sequence activates the motor pin, the virtual relay toggles, producing an audible click in the simulation interface. This click event is observable in the Wokwi simulation timeline and serves as a functional verification of the motor timing logic in the absence of a real actuator.

The LDR sensor and trigger switch are directly supported by Wokwi and behave identically to their physical counterparts. The simulation allows interactive toggling of the trigger switch and light level of the LDR to test all four combinations of trigger and sensor states, verifying that the serial output strings match expected values. This simulation environment is particularly valuable for validating the runMotor() timing logic before committing to a physical build.

```
void setup() {
  // Initialize pins
  pinMode(sensor, INPUT);
  pinMode(trigger, INPUT_PULLUP); // We should use pinMode(trigger, INPUT); in real life because our switch module already has a resistor.
  pinMode(motor, OUTPUT);

  // Initialize serial communication
  Serial.begin(9600);
  while(!Serial); // Wait for serial connection to establish
}
```





11. Challenges and Hurdles

11.1 Ambient Light Interference

The most significant challenge in this project is the sensitivity of the LDR detection mechanism to ambient lighting conditions. Sunlight or overhead room lighting can introduce sufficient background illumination to keep the LDR sensor in a persistent HIGH state, causing false hit detections independent of screen content. This was mitigated by calibrating the LDR module's comparator potentiometer to a threshold that reliably distinguishes the screen flash from ambient light levels in the intended playing environment. A design improvement for future iterations would be to use a narrowband optical filter over the LDR, reducing sensitivity to ambient light while maintaining response to the monitor's white flash.

11.2 Serial Communication Synchronisation

Establishing reliable synchronisation between the Arduino's continuous serial output and Unity's reading logic presented a notable challenge. If Unity reads a partial message or the buffer contains stale data from a previous cycle, false states can be parsed. This was addressed by using Serial.println() on the Arduino side, which appends a newline delimiter, allowing Unity to read complete line-terminated messages rather than raw bytes. Additionally, the delay(1) at the end of the Arduino loop prevents data flooding that could overflow the serial buffer.

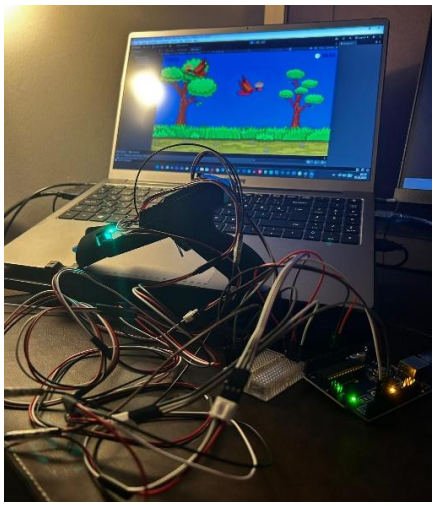
11.3 Motor Timing Without Real-Time Clock

Implementing a precise motor duration without using millis() required careful calibration of the eTime threshold values. Since eTime counts loop iterations rather than milliseconds, any variation in loop execution time — caused by serial transmission delays, for example — slightly alters the actual motor duration. During testing, the threshold values of 50 and 80 were empirically tuned to produce a vibration duration that felt natural as haptic recoil. A millis()-based approach would provide more deterministic timing and is recommended for future revisions.

11.4 Buffer Size and Data Formatting

The three-character buffer allocated for the sprintf() output string is borderline in size. While it functions correctly for the current single-digit binary values, it does not include space for the null terminator, which is written to str[3], a location one byte beyond the declared array size. This represents a latent buffer overflow bug. The fix is straightforward — declaring str[4] — and should be applied in any production version of the firmware to ensure memory safety.

12. Conclusion



The Duck Hunt Game project successfully demonstrates the integration of embedded hardware and real-time game software into a coherent interactive system. By combining an Arduino Uno's precise digital input reading and output actuation capabilities with Unity's rendering and game logic environment, the project recreates a classic gaming experience while simultaneously serving as a practical demonstration of hardware-in-the-loop design principles. The sequential light detection mechanism — adapted from the original 1984 NES Zapper — proves that a simple LDR sensor and a carefully timed screen flash can achieve reliable targeting accuracy at low cost and complexity.

Through the development of this system, key embedded systems competencies were exercised and reinforced: digital pin configuration and reading, logical signal inversion for pull-up resistor interfacing, serial UART communication at a defined baud rate, implicit state machine design using a loop-iteration counter, and H-bridge motor driver control for actuator feedback. The comparison between delay(), millis(), and formal FSM approaches highlighted the design trade-offs inherent in embedded timing, and the chosen hybrid approach was justified within the constraints and requirements of the application.

Several avenues for future improvement exist. Replacing the delay(1)-based timing with a millis()-based FSM would yield more deterministic motor control timing. Resolving the sprintf() buffer overflow vulnerability would improve firmware safety. Adding a narrowband optical filter to the LDR module would reduce ambient light sensitivity and expand the range of usable lighting conditions. On the software side, implementing a configurable COM port selector in Unity's ComController would improve usability across different PC configurations. Finally, extending the game with multiple target

types, difficulty progression, and sound effects would enhance the overall user experience while remaining fully supported by the existing hardware architecture.

In summary, this project achieves its intended objectives and serves as a compelling demonstration of how a low-cost microcontroller can function as a precise, responsive hardware peripheral within a complex interactive software environment — a paradigm that underpins a wide range of modern embedded product designs.

13. References

[1] Nintendo R&D1, "Duck Hunt," Nintendo Entertainment System game, Nintendo, 1984.

[2] Arduino, "Arduino UNO Rev3," Arduino Official Documentation, [Online]. Available: https://docs.arduino.cc/hardware/uno-rev3/. [Accessed: Apr. 2026].

[3] Arduino, "Serial communication reference," Arduino Reference, [Online]. Available: https://www.arduino.cc/reference/en/language/functions/communication/serial/. [Accessed: Apr. 2026].

[4] Unity Technologies, "Unity 2D game development," Unity Manual, [Online]. Available: https://docs.unity3d.com/Manual/Unity2D.html. [Accessed: Apr. 2026].

[5] ASIC Corp., "L9110S dual H-bridge motor driver datasheet," [Online]. Available: https://www.elecrow.com/download/datasheet-l9110.pdf. [Accessed: Apr. 2026].

[6] Wokwi, "Duck Hunt Arduino simulation," Wokwi Online Circuit Simulator, 2026. [Online]. Available: https://wokwi.com/projects/460911006135012353. [Accessed: Apr. 2026].

[7] K. Şahin, "KagansDuckHuntGame," GitHub, Apr. 2026. [Online]. Available: https://github.com/KaganxSahin/KagansDuckHuntGame. [Accessed: Apr. 2026].

[8] Atmel Corporation, "ATmega328P 8-bit AVR microcontroller datasheet," Rev. 7810D, Microchip Technology, 2015. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf. [Accessed: Apr. 2026].